RouteBalance: Fused Model Routing and Load Balancing for Heterogeneous LLM Serving

Wei Da
University of Cambridge
United Kingdom

Evangelia Kalyvianaki
University of Cambridge
United Kingdom

Abstract

Heterogeneous LLM serving stacks split scheduling into two layers that optimize in isolation: model routers pick a model from quality and cost signals while ignoring instance load, and serving load balancers optimize queues while ignoring quality. We present **RouteBalance**, a serving-aware scheduling layer that fuses both into a single online assignment over concrete model *instances*, jointly trading off quality, latency, and cost. A batched in-process predictor stack and dead-reckoned instance state keep the joint decision cheap on the request hot path (≈32 ms at 12 req/s). On a 13-instance, 28-GPU heterogeneous cluster serving four model sizes, a *single deployed* RouteBalance stack traces the upper region of the three-way quality–cost–throughput frontier. Sweeping one weight vector reaches both the highest routing-decision quality (DeepEval 0.419, +0.013 over the strongest baseline, 95% CI [+0.005, +0.022]; the ordering holds when a second judge re-scores the actually served text) and, at its cost-priority corner, per-request cost that ties the cheapest baseline. With router engineering equalized against concurrent-scoring baseline variants we build, its balanced preset serves at 2.8 s at 30 req/s—2.6–4.1× ahead of enhanced BEST-Route at high load. (Deploying those routers *as published*—one serial scoring call per request—makes them collapse 23× under load, a deployment-architecture effect we isolate separately, not the routing result.) A four-arm isolation shows the benefit follows from *pricing latency at model-selection time*; the learned predictors contribute calibration and SLO headroom rather than the headline frontier. Code: https://github.com/AKafakA/route-balance.

Keywords: LLM serving, request scheduling, model routing, load balancing, heterogeneous GPU clusters, quality-aware serving

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Cloud providers increasingly serve large language models (LLMs) on *heterogeneous* infrastructure, deploying models of different sizes across diverse GPU types to balance cost, latency, and output quality. A single cluster may answer complex queries with large models on powerful GPUs and simpler ones with small models on cheaper GPUs, each pairing offering a different mix of throughput, latency, and quality. This creates a scheduling question existing systems answer only in pieces: *which model instance should serve each request?*

The three factors interact. **Quality** is workload-dependent: larger models are generally better, but on simple queries a small model can match or beat a larger one [5, 8, 38]. **Cost** depends on prompt and output lengths and per-token price; larger models charge more per token but often answer more concisely. **Latency** depends on instance load, model size, lengths, and hardware, so scheduling must price latency at *model-selection* time—the term existing routers omit.

The cost of ignoring load is concrete. On our cluster, a quality-only router that always picks the nominally best model for each prompt drives mean end-to-end latency from 2.3 s to over 60 s as arrival rate rises to 30 req/s, because it concentrates traffic on a few high-quality replicas while cheaper tiers sit idle; conversely, a load-only balancer that ignores quality forfeits +0.04–0.05 DeepEval quality it could have kept at the same latency. Neither layer alone can occupy the good region of the trade-off.

Existing request management falls into two largely disjoint camps. *Model routers* [8, 29, 36, 38] choose a model from quality and cost signals but treat serving capacity as static and ignore instance-level load: a router may pick the nominally best model even when its instances are saturated. *Serving schedulers* [13, 46] optimize placement within a replica pool, improving latency for identical replicas, but do not reason about quality or cost across model sizes. Pre-LLM model-variant selectors (INFaaS [45], Cocktail [21], Tabi [52]) assumed stateless predictors and per-variant fixed costs; in LLM serving, cost and latency depend on *generated output length*, per-instance latency on decode state, and quality on the prompt. To our knowledge no LLM serving system jointly considers all three across a heterogeneous multi-model cluster.

RouteBalance closes this gap by formulating routing as **online assignment over concrete model instances** rather

than over model names. For each batch of waiting requests it solves a weighted quality–latency–cost score over request–instance pairs, with tunable 3-simplex weights exposing named operating points (Quality, Latency, Cost). The decision is made cheap by amortizing prompt-dependent estimation across a batch—a single CPU-resident embedding feeds a k=10 KNN head returning per-model quality and expected output length—and by in-process per-tier latency heads, while a greedy longest-processing-time (LPT) pass with dead-reckoned instance state avoids herding on stale load signals.

The components are individually standard (sentence embeddings, KNN, gradient-boosted latency heads, LPT ordering); the contribution is the *serving-aware formulation* and a causal account, on architecture-controlled and engineering-equalized baselines, of *where* the benefit comes from. The durable finding is that pricing a latency term in model selection *at all* is what occupies the frontier: a four-arm isolation (§6.3) attributes the gain to this cross-tier mix shift, which a decoupled quality/cost router cannot make. Within a tier, reactive queue depth suffices and the learned predictors are calibration and SLO headroom, not load-bearing. This characterizes *when learning matters* in serving-aware routing.

Contributions. *(1)* Serving-aware LLM routing as instance assignment, extending the quality–cost trade-off studied by routers to quality–latency–cost under dynamic load (§4). *(2)* An amortized, prompt-dependent estimator and a cheap greedy assignment that keep the joint decision off the critical path (§4, §5). *(3)* An end-to-end evaluation with fairness-controlled and engineering-equalized baselines on a 13-instance heterogeneous cluster (442 configurations, ≈1.5M requests), isolating the source of the gains (§6). We sweep RouteBalance against Avengers-Pro [59], BEST-Route [15], vLLM Semantic-Router [51], and passthrough selectors with round-robin, shortest-queue, and random dispatch; the decoupled paradigm leaves substantial headroom on all three axes, and at iso-quality RouteBalance reduces both latency and cost.

2 Background

Prefill processes the prompt and sets Time-To-First-Token (TTFT); auto-regressive **decode** sets Time-Per-Output-Token (TPOT) and dominates long generations. Continuous-batching engines with PagedAttention, such as vLLM [32], recompose batches at each decoding step and grow the KV cache dynamically. This improves utilization and throughput but *couples* each request's latency to the current co-batch composition, queue depth, and KV-cache pressure on its instance. Dispatching under dynamic load is therefore central to LLM serving, especially in heterogeneous clusters where different model–GPU pairings expose different latency, cost, and quality characteristics.

Heterogeneity trade-offs. At scale a provider hosts several model sizes across GPU types with different memory, bandwidth, and compute (Table 1), creating coupled trade-offs that no single layer resolves. *Quality vs. cost*: larger models are generally better but not uniformly—a 3B model can match a 72B on a simple factual query while trailing badly on hard reasoning—so a quality-aware scheduler can route easy prompts to cheap tiers without hurting user-visible quality. *Cost vs. latency*: cheaper tiers are not always faster—the 14B on V100×4 has lower TPOT than the 7B on A30×1 despite a higher per-token price—so cost-minimizing and latency-minimizing routing disagree. *Load vs. quality*: sending every hard prompt to the best model concentrates traffic on its few replicas, so quality and serving latency trade off through queueing. Today these are addressed in isolation; the missing layer is *global batch orchestration*—scheduling batches across the whole heterogeneous cluster while jointly weighing request difficulty, model quality, serving latency, and cost.

3 Related Work and Motivation

LLM serving systems. vLLM [32] introduced PagedAttention with continuous batching. DistServe [64] disaggregates prefill and decode for goodput; Mooncake [41] extends disaggregation with a KVCache-centric architecture and prediction-based early rejection; QLM [40] manages admission queues for per-request SLOs; SLOs-Serve [9] and PolyServe [65] support multiple SLO types at once; BucketServe [62] groups requests by input length for batching efficiency; HyGen [47] co-locates latency-sensitive online and throughput-oriented offline work via latency prediction. These optimize *within* a single model's replica set; RouteBalance operates at the cluster level *across* models and GPU types—it decides *where* to send each request while such systems optimize *how* to batch it once it arrives, a complementary layer.

Heterogeneous LLM serving. HexGen [27] and Helix [37] serve a single LLM across heterogeneous GPUs by optimizing tensor/pipeline parallelism; HexGen-2 [28] adds prefill–decode disaggregation; ThunderServe [26] targets cost-efficient heterogeneous cloud deployment; FineServe [6] handles precision heterogeneity; and MILP-based planners [25] cut cost across mixed GPU types. These address *hardware* heterogeneity for a single model; RouteBalance addresses both hardware *and* model heterogeneity and adds quality-awareness to the scheduling decision.

LLM model routing. Routers direct queries to models from predicted quality and cost: FrugalGPT [8] cascades cheap-to-expensive; RouteLLM [38] learns binary strong/weak routing; Universal Model Routing [30] generalizes to N models; KNN-based routing [34] matches learned routers at lower sample complexity; PORT [55] and OmniRouter [36] add aggregate token-budget control; PILOT [39] casts routing as a

budgeted contextual bandit; dynamic quality–latency routing [4] targets edge networks. These select models but are oblivious to instance-level load and queue state. RouteBalance integrates routing *into* the scheduler, so model selection accounts for real-time instance state. Pre-LLM model-variant selectors (INFaaS [45], Cocktail [21], Tabi [52]) chose variants under accuracy/latency/cost constraints, but with stateless predictors and per-variant fixed costs—assumptions LLM serving breaks, since cost and latency depend on generated output length and decode state.

Output-length prediction and budget control. Length prediction underpins cost estimation: S^3 [63] predicts response lengths with an auxiliary LLM; PARS [48] approximates shortest-job-first via pairwise length ranking; TimeBill [18] predicts length and execution time to enforce time budgets. Point estimates carry high error, so RouteBalance uses its KNN-predicted length as an *average-case* admission filter (Eq. 2) and enforces the worst case at dispatch (a `max_tokens` clamp plus a streaming early-stop) rather than trusting predicted-length×TPOT alone. Generation-time controls are complementary: BudgetThinker [53] inserts budget tokens during inference and BeLLMan [44] signals applications to shorten outputs under congestion, but both require model or infrastructure changes, whereas RouteBalance works with unmodified models by deciding before generation.

Batch and request scheduling. Cloud batch scheduling—cost-minimizing provisioning in Eva [7], co-location-aware placement [35]—inspires RouteBalance's per-batch assignment. Within LLM serving, Staggered Batch Scheduling [50] schedules requests within a batch for *intra-instance* pipeline efficiency, whereas RouteBalance batches for *inter-instance* multi-objective routing, exploiting batch-level properties unavailable otherwise: per-signal normalization across candidates, LPT ordering for cluster makespan, and thundering-herd avoidance via sequential assignment with local state updates. Request-ordering work—prefix-aware scheduling [3], length-aware L4 [58], k-LPM under prefix-reuse constraints [14], accuracy-scaling Proteus [2]—is orthogonal and composes with RouteBalance's LPT-based greedy pass.

Latency and execution-time prediction. Three approaches predict LLM latency. *Simulation*—Vidur [1], LLMServingSim [11], Block [13], SamuLLM [19]—mirrors batching with profiled kernel times but needs per-configuration re-integration. *Analytical/roofline* models [57] estimate from lengths and hardware but assume static TPOT, which varies under load. *Learned* predictors train on observed state→latency data [18, 56], an increasingly shared foundation for serving optimizations [60].

Table 1. Heterogeneous routing pool: four Qwen2.5 models on three GPU types, with measured TPOT, throughput, and public per-token price.

Model	GPU	Inst.	TPOT	Tput	Price (USD/1M)
Qwen2.5-72B	A100×4	2	41.6	24	0.38/0.40
Qwen2.5-14B	V100×4	3	13.9	72	0.15/0.15
Qwen2.5-7B	A30×1	5	19.6	51	0.07/0.07
Qwen2.5-3B	A30×1	3	10.2	98	0.06/0.06

RouteBalance follows the learned approach—per-tier gradient-boosted TPOT heads, inspired by the learned-index paradigm [31], combined analytically with live decode state—chosen for forward-compatibility across heterogeneous tiers without the per-hardware re-profiling simulators require.

Table 1 summarizes our heterogeneous testbed: four Qwen2.5 models [42] (3B/7B/14B/72B) across three GPU types, with measured TPOT, throughput, and public per-token price. The heterogeneity entangles the three axes: larger models are not uniformly better per prompt; lower-price tiers are not always faster (14B on V100×4 has lower TPOT than 7B on A30×1 despite a higher price); and cost-aware routing itself shifts queueing onto cheap tiers. The missing layer is *global batch orchestration* across the cluster that jointly weighs quality, cost, and serving latency—the gap RouteBalance fills.

4 System Design

RouteBalance sits between clients and a heterogeneous serving cluster. Clients send ordinary generation requests, each optionally carrying a per-request cost budget; for every request RouteBalance picks a concrete model *instance* using prompt-dependent quality and length estimates together with a latency term in model selection. We take the instance set I and the scheduling weights as fixed at runtime and focus on the per-request assignment, which makes RouteBalance orthogonal to—and composable with—deployment-time planners that decide how many replicas of each model to place across the cluster [27, 37].

4.1 Batch scheduling and the assignment

RouteBalance schedules in batches: each batch is the set of requests waiting when the scheduler fires, and it assigns every request to an instance before the next batch starts. Batching gives four properties: predictors run once per batch (fixed cost amortized); scoring normalizes latency and cost within the batch as load changes; each in-batch dispatch updates the scheduler's local view of the chosen instance, so later requests avoid herding; and the batch can be ordered by predicted output length, longest first, following LPT [20]. Underlying this is a separation of two signal types. Quality and length are *prompt-intrinsic*, so the batched estimator computes them once per batch and reuses them across

all candidate instances; latency is *state-dependent*, so dead reckoning re-derives it per dispatch from the instance state. Batch size is adaptive—larger when more instances are busy, smaller when idle.

For a batch R_B over instances I, each request selects one instance, so the objective decomposes per request and the deployed mechanism is a *greedy batched procedure*: requests are scored in LPT order and each dispatch updates the dead-reckoning state seen by the next. Greedy is the natural choice because the assignment is *state-dependent*. Dispatching r to i raises i's queue depth and changes the latency estimate for every later request in the batch, an assignment that is NP-hard in general. RouteBalance therefore orders the batch by predicted output length, longest first (Graham's LPT rule, whose 4/3 makespan guarantee [20] motivates the ordering), using $\hat{L}_r = \max_m \hat{L}_{r,m}$ as the sort key since the model is not yet chosen. It then dispatches greedily, updating the dead-reckoning state after each assignment. This runs in $O(|R_B|\,|I|)$ rather than the exponential cost of exhaustive matching. A batch-level matching (e.g. Hungarian) would differ only through within-batch state updates, and an offline replay over the logged score matrices changes 15.6% of assignments but leaves realized quality unchanged (−0.002), so the greedy gap is empirically negligible. Each greedy step maximizes

$$S_{r,i} = w_{\text{qual}}\,\hat{Q}_{r,m(i)} + w_{\text{cost}}\left(1 - \frac{\hat{C}_{r,i}}{\max_j \hat{C}_{r,j}}\right) + w_{\text{lat}}\left(1 - \frac{\hat{T}_{r,i}}{\max_j \hat{T}_{r,j}}\right) \tag{1}$$

subject to one instance per request, with weights on the 3-simplex ($w_{\text{qual}}+w_{\text{cost}}+w_{\text{lat}}=1$). Quality enters as the raw KNN score $\hat{Q}_{r,m(i)} \in [0,1]$; cost and latency are per-request normalized by their candidate maxima, so a 6×-cheaper candidate contributes proportionally rather than 0/1. Normalization is essential because the three signals have incomparable scales: a quality score in $[0,1]$, a cost in fractions of a cent, a latency in seconds. The batch supplies the candidate set against which each request's cost and latency are scaled, a reference a point-at-a-time router (scoring one request with no batch context) does not have. The term model-only routers lack is any latency term in model selection; its deployed form is the live estimate $\hat{T}_{r,i}$. Table 2 summarizes the notation.

If a request supplies a budget b_r^{cost}, a pre-scoring filter keeps only candidates whose predicted total cost (input tokens plus KNN-predicted output length, at the model's rates) fits within it:

$$\hat{C}_{r,i} = \ell_r^{\text{in}} c_{m(i)}^{\text{in}} + \hat{L}_{r,m(i)} c_{m(i)}^{\text{out}} \le b_r^{\text{cost}}. \tag{2}$$

This is an average-case admission filter; the worst case is enforced at dispatch (`max_tokens` clamped to the remaining budget) and by a streaming early-stop when running cost exceeds b_r^{cost}.

Table 2. Scheduling objective notation.

Symbol	Meaning
R_B, I, $m(i)$	batch, instances, model served by instance i
$\hat{Q}_{r,m}$, $\hat{L}_{r,m}$	predicted quality and output length for r on m
$\hat{T}_{r,i}$	predicted end-to-end latency for r on instance i
$\hat{C}_{r,i}$	expected serving cost (input+predicted-output tokens at m's prices)
w_{qual}, w_{lat}, w_{cost}	3-simplex weights ($\sum = 1$)
b_r^{cost}	optional per-request cost budget

Algorithm 1 RouteBalance per-batch greedy dispatch

Require: batch R_B, instances I, weights $(w_{\text{qual}}, w_{\text{lat}}, w_{\text{cost}})$
$E \leftarrow \textsc{EmbedBatch}(R_B)$ ▷ one MiniLM call
$(\hat{Q}, \hat{L}) \leftarrow \textsc{KnnLookup}(E)$; $\hat{T}^{\text{tpot}} \leftarrow \textsc{TierHeads}()$
$D \leftarrow \textsc{ReadTelemetry}(I)$ ▷ seed dead-reckoning state
for $r \in \textsc{SortByPredLenDesc}(R_B)$ **do** ▷ LPT order
  $C \leftarrow \{i \in I : \hat{C}_{r,i} \le b_r^{\text{cost}}\}$ ▷ budget filter
  $i^\star \leftarrow \arg\max_{i \in C} S_{r,i}$ ▷ Eq. 1, using D
  dispatch $r \to i^\star$; $\textsc{Update}(D, i^\star, \hat{L}_{r,m(i^\star)})$
end for

Algorithm 1 states the per-batch procedure. The embed-and-estimate step is one batched call shared across the batch; the per-request loop is vectorized arithmetic over precomputed per-tier predictions, and each dispatch updates only the chosen instance's local dead-reckoning state, so the next request in LPT order sees a current view without a fresh telemetry round-trip.

4.2 Estimators

Model estimator. Per batch the scheduler embeds all prompts in one batched call to the CPU-resident `all-MiniLM-L6-v2` encoder, then queries a distance-weighted FAISS [16] KNN index (k=10) over the 14,919-prompt training split. Each neighbor stores per-model quality labels and output lengths, so one lookup returns a predicted quality and expected length for every candidate. Quality labels are precomputed offline with DeepEval G-Eval against dataset references, so no judge runs on the request path. The precomputed per-(prompt, model) score is the routing-decision metric, the standard basis on which offline routing benchmarks are evaluated [22, 38]. The estimator exposes a *metric-agnostic* interface—it maps each prompt to a score in $[0,1]$ per candidate model regardless of how that score is computed—so an operator can swap the quality signal (LLM-judge score, reference-grounded accuracy, embedding similarity, code pass rate) through one configuration change. Treating quality and length as *prompt-intrinsic* model properties also matches the Model-as-a-Service abstraction, where billing depends on token usage and model choice rather than serving hardware;

this is what makes the offline per-(prompt, model) precompute valid. Crucially, both the offline grid and online serving decode *greedily* (temperature 0, frequency penalty 1.2), so a given (prompt, model) pair yields a deterministic response and the precomputed score is the score of the text actually served.

Latency heads. For each (model, GPU) tier we train one XGBoost [10] TPOT head from a tier-local QPS sweep on that tier's head node; at run time the scheduler queries every tier's head in parallel. End-to-end latency is estimated analytically: $\hat{T}_{r,i} = \hat{T}^{\text{tpot}}_{r,i} \cdot \left(d_i/b_i + \hat{L}_{r,m(i)}\right)$, where d_i is the instance's pending decode tokens and b_i its current decode batch size, so d_i/b_i is the iterations the request waits through before its own $\hat{L}$ decode steps; if the instance has a free decode slot only the second term applies. The per-batch cost is $O(|R_B|)$ embeddings plus $O(|R_B|\,|I|)$ vectorized score evaluations—one XGBoost call per *tier* (not per instance), the rest precomputed-array arithmetic—so the embedding dominates at our $|I|$=13 and the design degrades gracefully with instance count (scoring-loop cost 12.8/14.3/22.5 μs at $|I|$=13/100/500).

We use learned per-tier heads rather than a kernel-level simulator or a roofline model for *forward-compatibility*. A simulator such as Vidur [1] must be re-integrated whenever the backend's batching logic changes (e.g. a vLLM engine revision). Analytical/roofline estimators [57] need ∼30 GPU-hours of per-hardware profiling and assume a static per-hardware TPOT constant, so they are neither forward-compatible to software updates nor backward-compatible to a new GPU type. A per-tier head instead adapts by retraining on a short tier-local QPS sweep. Consistent with the isolation of §6.3, this learned head is *not* load-bearing for the headline frontier—a static per-tier prior reproduces it—but it is the deployment default because it supplies calibration under drift and the residual-CDF an SLO admission filter would query.

Dead reckoning. Reading live telemetry once per batch is cheap, but within a batch the chosen instance's state changes after every dispatch. Rather than re-poll, RouteBalance updates a local copy of the chosen instance's decode state ($d_{i^\star}$ grows by $\hat{L}$) after each assignment, so subsequent requests in LPT order see the consequences of earlier ones and the batch does not herd onto whichever instance looked idle at batch-formation time. This local update is what makes the greedy pass a good approximation to a batch-level matching (the 15.6% assignment-divergence, −0.002 quality replay above).

4.3 Runtime

Figure 1 shows the runtime: receive requests, form a batch, read instance telemetry to seed the dead-reckoning state, run model estimation, then the LPT-based greedy assignment—sort by predicted length, score by Equation 1, and update the chosen instance's local state after every dispatch. All predictors run in-process behind one modular interface; the same runtime supports decoupled router/dispatcher baselines through "pipeline mode" (§5), so every system runs inside an identical scheduling and batching path.

5 Implementation

RouteBalance is ≈13,000 lines of Python. The scheduler runs as its own service next to one vLLM worker per (model, GPU) instance; each worker exposes a non-blocking telemetry endpoint (queue depth, pending decode work, active sequences, KV-cache pressure, recent service rate). All hot-path predictors live in the scheduler process pinned to CPU, so they never contend with worker GPU inference, and each fired batch issues predictor calls over the full $|R_B| \times |I|$ candidate matrix in one shot—the basis of the per-batch amortization. We use vLLM defaults except tensor-parallel flags for the 72B; nothing assumes vLLM specifically.

Native in-process predictors. The estimator and latency heads load into the scheduler process *pinned to CPU*, so they never interrupt or contend with the workers' GPU inference; even so, the booster keeps a TPOT query at ≈3 ms. The telemetry endpoint is non-blocking and returns immediately from a worker-side cache the inference loop refreshes, so seeding the dead-reckoning state never stalls decode. These two contracts—CPU-pinned predictors and non-blocking telemetry—are what let the joint decision sit on the hot path at ≈28–32 ms/request without slowing the backends.

Pipeline mode for baselines. The same service supports decoupled router/dispatcher baselines: Equation 1 is bypassed, a pluggable estimator first picks a model (Avengers-Pro p_w-mix, BEST-Route threshold, or passthrough), and a dispatcher (round-robin, shortest-queue, or random) places the request within that model's replica pool. Every baseline therefore runs inside RouteBalance's own batching, telemetry, and dispatch path; only the routing logic and dispatcher differ across rows. This makes the comparisons *architecture-controlled* rather than confounded by implementation differences, and it is also how we build the *enhanced* concurrent-scoring baseline variants of §6.3: the same checkpoints with scoring moved off the scheduling loop.

6 Evaluation

We ask two questions: how does a single fused stack compare to decoupled router/dispatch baselines on the multi-objective frontier (§6.2), and *where* do its gains come from (§6.3)? We then validate budget control (§6.4), batching (§6.5), and robustness (§6.6–§6.9).

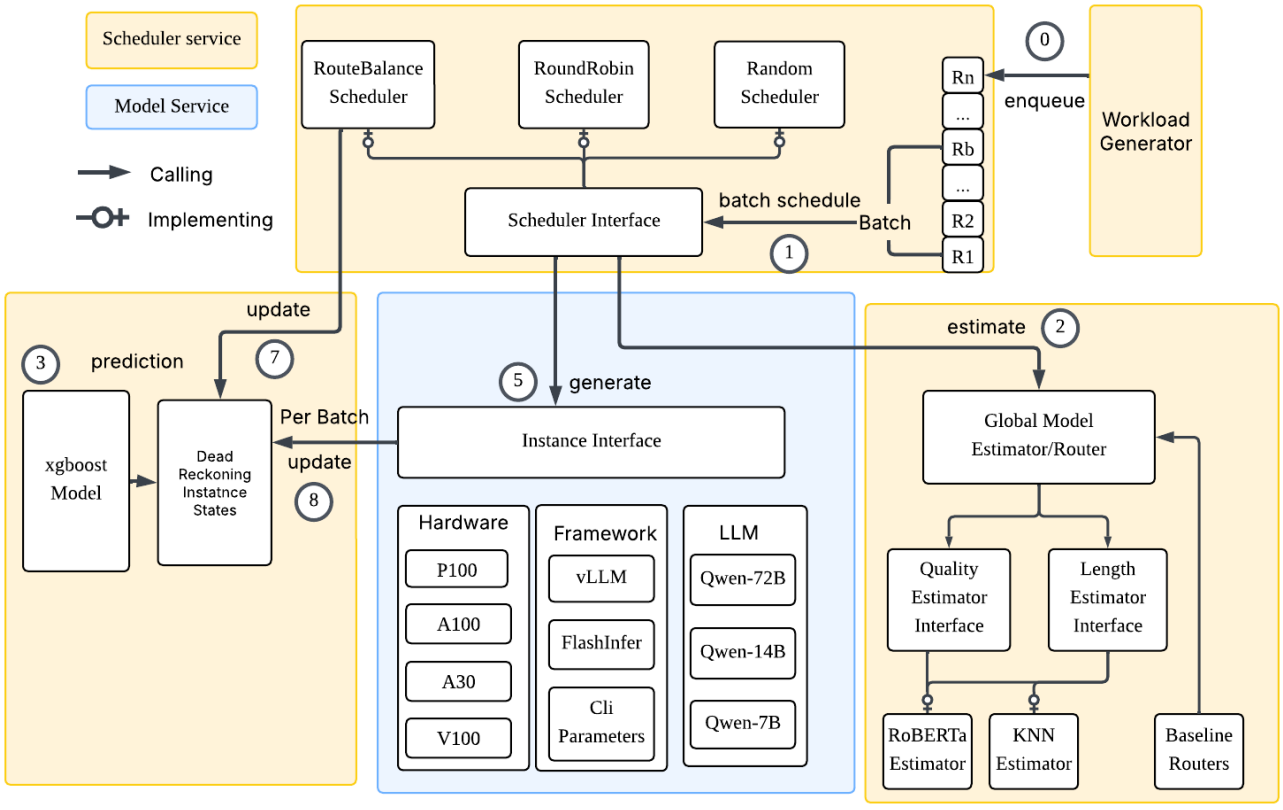

Figure 1. RouteBalance runtime architecture: batch formation, telemetry-seeded estimation, and LPT-greedy assignment over the $|R_B| \times |I|$ candidate matrix.

6.1 Setup

We drive the 13-instance cluster of Table 1 (72B and 14B use tensor parallelism TP=4) with the standard vLLM serving-benchmark harness [32] (the de facto LLM-serving evaluation tool in academia and industry) at cloudLab [17], replaying held-out test prompts at a fixed mean rate λ per cell under its Poisson arrival model (and, for §6.9, gamma-bursty and square-wave processes). We release a model-estimator dataset of 18,608 prompts, each broadcast to the four Qwen2.5 candidates, from seven public datasets covering instruction following, code, safety, multi-turn chat, math, and reading comprehension [12, 23, 24, 33, 43, 54, 61], split 80/20 into 14,919 train and 3,634 test (55 prompts dropped in scoring/filtering); serving cells use the 3,534-prompt subset with complete coverage. Quality is scored off-line with DeepEval G-Eval [49] judged by `Llama-3.1-8B-Instruct` (outside the Qwen pool, so no candidate grades itself) against dataset references. We report **quality** (DeepEval), **throughput** (completed req/s), **end-to-end latency** (mean completion), and **cost** (realized tokens at public per-token prices). **All trained routers consume identical supervision:** BEST-Route's DeBERTa-v3 scorer and Avengers-Pro's clusters are (re)fit on our train split using the same DeepEval labels the KNN estimator uses, so quality differences reflect architecture and policy, not supervision alignment, and no trained component sees the evaluation prompts.

We compare against Avengers-Pro ($p_w \in \{0.25, 0.39, 0.53, 0.70, 0.80\}$), BEST-Route (threshold $t \in \{0, 0.3, 0.5, 0.6, 0.7, 0.8\}$), each with round-robin and shortest-queue dispatch, and a passthrough router with all three dispatchers; vLLM Semantic-Router [51] runs as a separate-process baseline. RouteBalance sweeps 16 weight tuples on the simplex. In total 287 no-budget cells over $\lambda \in \{6..30\}$ plus budget, batching, multi-seed, vLLM-SR, and alternate-judge studies (442 configurations, ≈1.5M requests).

6.2 The frontier: one stack vs. baselines

Figure 2(a) snapshots λ=12 on the quality–latency plane: turning only the weight vector, RouteBalance traces a monotone frontier from its cost-priority preset (2.2 s, quality 0.354) through uniform (2.3 s, 0.371) and the mid-band (4.1 s, 0.397) to the quality ceiling at w_q=0.8 (5.9 s, **0.419**)—+0.013 above

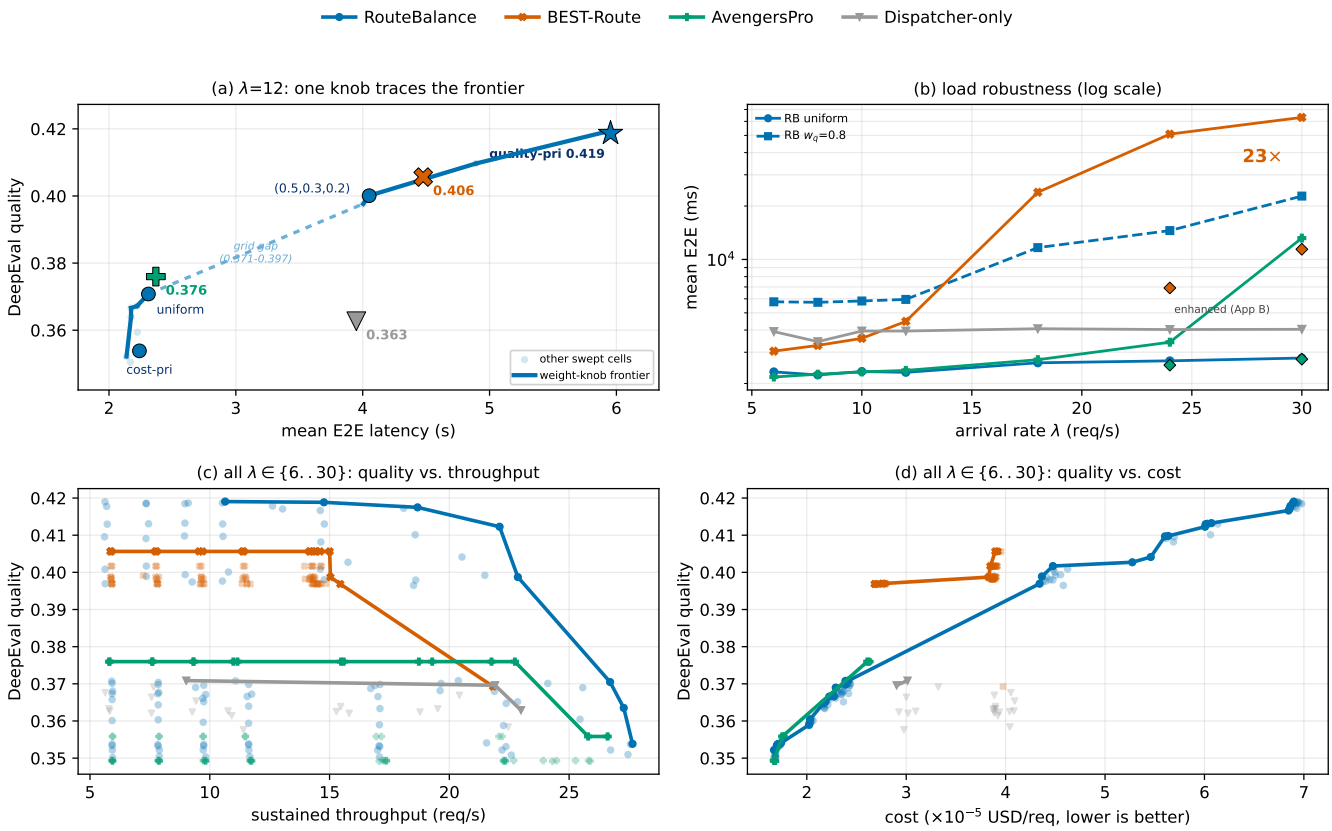

Figure 2. Quality–latency–cost trade-offs; color keys the four systems (§6.2). (a) quality–latency at λ=12; (b) mean E2E under load (RouteBalance presets: solid uniform, dashed w_q=0.8; log scale; diamonds = enhanced variants); (c) quality–throughput and (d) quality–cost hulls pooled over all λ (lower cost better).

the best BEST-Route cell (0.406) and +0.043 above Avengers-Pro (0.376). Both gaps are significant under a paired per-prompt bootstrap on the same 3,534 prompts ($\Delta_{\mathrm{RB-BR}}$=+0.013, 95% CI [+0.005, +0.022]; $\Delta_{\mathrm{RB-AP}}$=+0.043, [+0.033, +0.053]). The quality of record here is the routing-decision lookup (§4.2), the standard offline-routing basis; re-judging the *actual served text* of the headline pair under a second judge preserves the same ordering (§6.7), so the gap is not an artifact of matching the lookup table. Each baseline is one fixed point on or near the curve.

Panel (b) is the load-robustness view. With router engineering *equalized*—concurrent-scoring variants of both baselines that we build (routing and quality unchanged)—RouteBalance's uniform preset stays at 2.3–2.8 s across the sweep, 2.6–4.1× ahead of enhanced BEST-Route at λ=24–30 (6.9/11.4 s), while enhanced Avengers-Pro matches uniform's latency. Deployed as published (one scoring call per prompt), BEST-Route instead climbs from 4.5 s at λ=12 to 63 s at λ=30 (23× uniform's 2.8 s there; iso-quality 2.8×), and Avengers-Pro turns upward from λ=24. This is a deployment-architecture effect, which the ladder of §6.3 separates from policy. Panels (c) and (d) pool every valid cell with per-system upper hulls. On throughput (c) RouteBalance dominates the frontier (its best quality at least every baseline's at every sustained-throughput level, all 175 baseline cells) and reaches 27.6 req/s where BEST-Route tops out at 21.8. On cost (d) its hull spans from the cheapest served cost (1.67×10^{-5}, tying Avengers-Pro) up to the 0.419 ceiling, dominating Avengers-Pro and dispatcher-only at every cost; only inside the disclosed mid-cost grid gap (§6.3) is BEST-Route briefly competitive.

The defining property is a *single deployed stack* reaching every corner by changing only the weights: at w_q=0.8 the quality ceiling 0.419 (concentrating on 72B, 5.9 s, low throughput); at uniform weights 2.3–2.8 s across the whole load range; at the cost-priority corner the cheapest cost of any system (1.67×10^{-5} USD, tying Avengers-Pro, still serving every request; BEST-Route's cheapest is 2.68×10^{-5}; Table 3). Each baseline occupies one slice. Figure 3 makes this visual: the RouteBalance family reaches the rim on quality, cost, and mean latency at every λ, while Avengers-Pro holds the p99 rim only at low load (lost by λ=24) and BEST-Route collapses on both latency axes from λ=18.

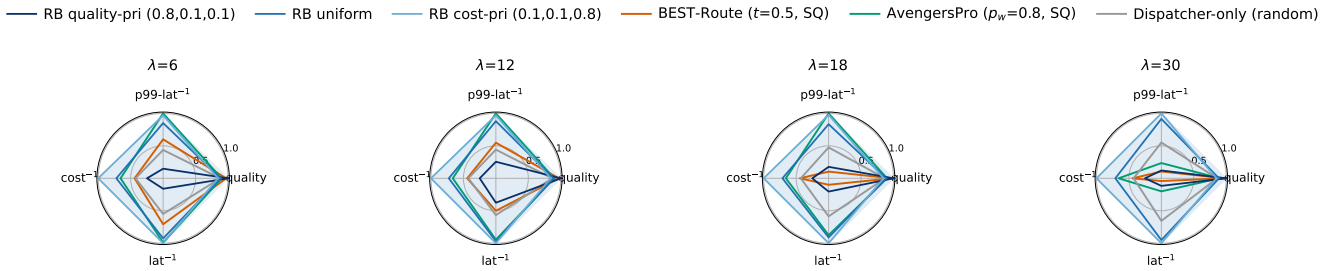

Figure 3. Per-λ capability radar ($\lambda \in \{6, 12, 18, 30\}$): blue outlines are three presets of the *same* RouteBalance deployment; each axis is the fraction of the best plotted cell at that λ (rim = best).

Table 3. RouteBalance vs. each baseline: per-axis best over the sweep, plus mean E2E under load; bold = best. †Enhanced rows use *our* concurrent-scoring (batching), not the baseline (§6.3).

System	Peak qual. (DQ)	Max tput (req/s)	Min cost (USD/req)	E2E under load	cells (hull)
RouteBalance	**0.419**	**27.6**	$\mathbf{1.67\times10^{-5}}$	2.3–2.8 s	112
BEST-Route	0.406	21.8	2.68×10^{-5}	11.4 s†	84
Avengers-Pro	0.376	26.6	1.67×10^{-5}	~2.3–2.8 s†	70
Dispatcher-only	0.371	23.0	2.90×10^{-5}	3.2–3.8 s	21

Why does one stack reach both ends of the frontier? The wrapper baselines are thresholding routers. A low threshold queue-bottlenecks the 14B/72B replicas (high quality, low throughput) and a high threshold floods 3B (the reverse). Intermediate thresholds trace a steep concave-down hull because the per-request decision is binary, with no within-batch retargeting under back-pressure. RouteBalance instead picks per-prompt *and* per-instance jointly: it keeps harder prompts on 14B/72B heads while filling 3B/7B capacity with shorter, easier ones (at w_q=0.8 mostly 72B; uniform spreads ~57% 3B / ~32% 14B). The whole mix comes from a *single* deployed stack with only the weights changing. Together these mechanisms give the peak quality +0.043 above Avengers-Pro—over 80% of the 0.052 always-3B→always-14B step (0.346 vs 0.398)—and +0.013 above BEST-Route, while at the cost corner the per-request normalization in Equation 1 lets a 6×-cheaper candidate pull the assignment without a threshold router's all-or-nothing flip. A four-seed study places RouteBalance at 0.4184±0.0003 with the deterministic baselines at *zero* cross-seed variance, so the ordering is stable across sampling-noise and run-to-run sources.

6.3 Where the benefit comes from

Scheduling overhead. Joint routing is cheap on the hot path: RouteBalance's mean per-request off-instance residual (client E2E minus instance-reported E2E—network plus all scheduler-side routing/queueing/batching) grows only sub-linearly, 129 ms at λ=6 to 231 ms at λ=30 (1.8× over a 5× load increase). Table 4 decomposes it: the MiniLM+KNN decision compute is the dominant ≈27 ms term and *decreases* with load (32.3→28.2 ms) via intra-batch amortization, the growth being batch-formation queueing—fully charged in every reported E2E, compensated downstream by a better global assignment rather than cancelled in place. The trained-router baselines show the opposite: comparable at $\lambda \leq 12$, they then saturate the per-request scoring queue—BEST-Route to 57.9 s residual at λ=30, Avengers-Pro's faster k-means later (258 ms→2.79 s). Passthrough's near-zero residual (36 ms) still gives *worse* E2E at λ=12 (3806 vs 2311 ms): a small residual is neither necessary nor sufficient.

Why the residual stays bounded: a vanishing batch bubble. The added off-instance wait a request incurs from batching is $wait \approx \max(0,\ t_{\text{form}} - t_{\text{busy}}) + t_{\text{compute}} + t_{\text{tele}}$, where t_{form} is the batch-formation window, t_{busy} is the queueing the request would face on its instance anyway, t_{compute} is the per-request share of the amortized decision, and t_{tele} is the telemetry RPC. Adaptive sizing ties t_{form} to cluster busyness, so the *batch bubble*—the term $\max(0, t_{\text{form}} - t_{\text{busy}})$, the idle waiting introduced purely by batching—collapses toward zero exactly when it would matter. Under saturation t_{form} is dominated by t_{busy} (the request was going to queue regardless), while at light load batches are small so t_{form} is small. This is why the residual rises only 1.8× over a 5× load increase while t_{compute} actually *falls* via amortization; a per-request router has no such bubble to amortize—its scoring queue grows monotonically with arrivals.

Table 4. RouteBalance off-instance-residual decomposition at uniform weights (N=3534/cell, ms). *Compute* sums the MiniLM+KNN estimator, XGBoost TPOT, and scoring; TTFT and E2E are client-observed (include the residual).

λ	compute	batch wait	stats fetch	**total residual**	client TTFT	E2E
6	32.3	48.0	12.4	**128.7**	160.2	2325
12	32.4	47.9	12.6	**130.0**	161.7	2311
18	29.1	51.7	11.2	**169.3**	205.6	2615
24	28.3	57.6	10.1	**181.5**	218.4	2684
30	28.2	101.2	10.4	**230.6**	269.8	2783

Table 5. Off-instance residual vs. end-to-end latency (per-request means, ms, N=3534/cell). RouteBalance at uniform weights; baselines at their peak-quality cell, round-robin dispatch; *enhanced* rows are our concurrent-scoring variants (§6.3).

	λ=12		λ=24		λ=30	
System	residual	E2E	residual	E2E	residual	E2E
RouteBalance (uniform)	130	**2311**	181	**2684**	231	**2783**
AvengersPro pw=0.8, RR	258	2574	812	3773	2793	7205
enhanced (ours, SQ)	72	2301	118	2538	222	2741
BEST-Route t=0.5, RR	697	4289	42408	49491	57948	64204
enhanced (ours, SQ)	194	3445	569	6908	2399	11416
Passthrough, RR	36	3806	44	3918	55	3930

A deployment ladder. A natural objection is that BEST-Route's collapse reflects how its router is *deployed*, not its policy. We tested the full ladder (Table 5). *(i) Serial scoring*, the shipped pattern, caps throughput near the forward rate (431 ms/prompt single-threaded), producing the 49.5–64.2 s collapse at λ=24–30. *(ii) Micro-batched but co-located* re-runs reach 214/238 s at λ=24/30, 98% router-side queueing. *(iii) vLLM-SR*, an untouched external system whose content-aware classifier runs as a separate CPU service, collapses from λ=18 the same way (Table 6)—independent evidence (i) is representative, not a strawman. *(iv) An enhanced variant we build*, the same BEST-Route checkpoint with scoring moved to concurrent execution off the scheduling loop, survives the sweep (3.0–11.4 s, routing byte-identical to serial) but still trails RouteBalance's uniform preset 2.6×/4.1× at λ=24/30; the remaining gap is instance-blind routing pushing 98% of traffic onto the three-instance 14B tier regardless of load (measured at the t=0.5 headline configuration; the concentration, and hence the magnitude, depends on the threshold and this cluster's tier sizing). The engineering that makes rung (iv) work also explains the gap to rung (ii). Scoring runs on the default thread-pool executor (32 threads on the 96-core scheduler node), so at λ=30 and 431 ms/forward roughly 13 concurrent forwards ($\approx$ 14% of cores) each pay their *own* prompt's length. The co-located micro-batch collector instead pads every batch to its longest sequence (1.72 s per batch of 64 at 256 tokens) and cannot overlap batches. The same checkpoint is therefore fast concurrently and slow when padded. The same enhancement applied to AvengersPro removes its upturn (2.18–2.74 s); against it, RouteBalance's advantages are the quality ceiling, the cost tie, and the weight family, not headline-cell latency. The ladder's reading: amortized batch scoring is a design requirement, since three independent per-prompt deployments hit the same wall, and RouteBalance meets it by construction ($\approx$28 ms/request co-located). With engineering equalized, the comparison becomes policy-vs-policy, which joint instance-aware assignment wins.

Table 6. vLLM Semantic-Router head-to-head (N=3534/cell; §6.3).

λ	completed	failed	quality	E2E
6–10	3534	0	~0.37	4.4–5.3 s
12	3260	274 (7.7%)	0.37	14.2 s
18	373	3161 (89%)	0.37	236 s
24	208	3326 (94%)	0.37	302 s
30	123	3411 (97%)	0.35	344 s

A four-arm isolation. To locate the source of the gains we re-serve the uniform cell in four arms at identical seed and prompts. *Arm 1*, the full objective; *arm 2*, w_{lat}=0 with the scheduler's reactive shortest-queue tiebreak; *arm 3*, w_{lat}=0 with a predictive $\hat{T}$-argmin tiebreak; *arm 4*, the full objective with $\hat{T}$ replaced by a *static per-tier prior* (nominal TPOT × predicted length; zero telemetry). Three findings (Table 7). *(i) Within a tier, prediction adds nothing over reactive queue depth*: arms 2/3 are a wash (+2.8/−0.7/−3.5% E2E). *(ii) The value is cross-tier*: pricing latency in the model score (arm 1) is 26–31% faster than decoupled-predictive routing by steering traffic off the slow 72B tier (14%→1%)—a mix shift a decoupled quality/cost router cannot make; the small quality decrement (0.369 vs 0.385) is the intended exchange. *(iii) The latency signal need not be learned or live*: a static per-tier prior (arm 4; nominal TPOT × length, zero telemetry) reproduces arm 1 with mix and quality unchanged—λ=18 2.41 vs 2.61 s (8% lower) and a 40/6 overload 3.12 vs 3.79 s (18%). The learned predictor is therefore *not* load-bearing for the headline frontier: what the baselines lack is a latency term in model selection at all. We retain the learned configuration as the deployment default (the prior is distilled from its traces; calibration under drift; SLO headroom).

The shaping is two-level. The time-averaged tier mix is set by the weight vector and is *rate-independent*: at fixed uniform weights the per-tier request shares are essentially constant across λ (from λ=6 to 30: 3B 56–58%, 14B ~32%,

Table 7. Isolation arms 1–3, uniform cell, mean E2E (s), N=3,534/cell; arms 2/3 share weights and mix. Arm 4 (static prior) is compared to arm 1 at λ=18 and a 40/6 overload in the text.

	mean E2E (s)			72B	qual.
	λ12	λ24	λ30	share	(λ12)
1. Full objective	**2.37**	**2.60**	**2.78**	1%	0.369
2. w_{lat}=0, reactive queue	3.33	3.53	3.89	14%	0.385
3. w_{lat}=0, predictive $\hat{T}$	3.42	3.50	3.75	14%	0.385

Table 8. Budget-exhaustion rate and DeepEval quality on the *actual served text* at λ=16, N=3,534/cell, over three budget-tightness mixes.

	Tight (75%)		Med. (45%)		Loose (30%)	
	exh.	qual.	exh.	qual.	exh.	qual.
RouteBalance+filter	**32.8**	**.233**	**19.2**	.290	**12.6**	.315
RouteBalance, no filter	39.1	.218	23.5	.278	15.5	.309
BEST-Route argmax	41.5	.226	25.4	.294	16.7	.330

7B ∼11%, 72B pinned at 1%). The weights thus fix a load-independent bias toward latency-efficient tiers. Within that bias, moment-to-moment instance selection still tracks instantaneous state—via $\hat{T}$ or, equivalently by finding (i), reactive queue depth. This is exactly what extends the result to non-stationary arrivals (§6.9): a rate-stable mix that already avoids the slow tier in every load phase, with queue-aware dispatch absorbing within-phase spikes.

6.4 Budget control

We measure budget-aware execution at λ=16, sweeping mixes with 75/45/30% of prompts budget-constrained. All three configurations share the runtime cap (dispatch-time clamp plus streaming early-stop): RouteBalance with and without the admission filter, and BEST-Route argmax without it. The within-system result is paired on identical prompts: the admission filter cuts exhaustion by 6.3 pp at the tightest mix (2.9 at the loosest) and converts that directly into quality at every mix (+0.015/+0.012/+0.006; Table 8), by routing to a cheaper model that completes rather than a larger one truncated to near-empty. The cross-system comparison is budget-regime dependent: RouteBalance+filter beats BEST-Route argmax at the tightest mix (0.233 vs 0.226); at looser mixes BEST-Route's larger models win given headroom. The mechanism—admission-time filtering converting exhaustion into quality on any router—is the contribution, not the simplex weights.

Table 9. Per-prompt bootstrap 95% CI on the peak-quality cell of each system at λ=12 (10,000 resamples, percentile method).

System	mean DeepEval	95% CI
RouteBalance (w_q=0.8)	**0.4187**	[0.4089, 0.4288]
BEST-Route t=0.5	0.4056	[0.3958, 0.4154]
AvengersPro p_w=0.8	0.3760	[0.3661, 0.3859]
Dispatcher passthrough	0.3627	[0.3531, 0.3729]

6.5 Batching ablation

Figure 4 ablates the three batching choices at uniform weights. LPT-off matches the default within ±2.3% E2E (dead-reckoning already steers off saturated instances); adaptive-off costs 0.4–6.0%, growing with λ. Fixed batch size: the batched-KNN estimator keeps bs=1 from collapsing (2.4/4.0 s at λ=16/24); bs=16/32 stay within ∼3.7% of the adaptive default.

6.6 Quality confidence and seed stability

We quantify uncertainty on the headline quality numbers two ways. A per-prompt paired bootstrap (10,000 replicates on the per-prompt quality difference over the same served prompts) places every RouteBalance–baseline gap above zero: +0.013 [+0.005, +0.022] vs BEST-Route, +0.043 [+0.033, +0.053] vs Avengers-Pro. The per-system peak-quality cells and their bootstrap intervals are in Table 9. A multi-seed study (four independent Poisson-arrival seeds; Table 10) finds the deterministic routers at *exactly* zero quality variance and RouteBalance stable to ±0.0004, with per-request cost token-based and hence seed-stable; the ordering RouteBalance > BEST-Route > Avengers-Pro is not a single-run artifact.

Re-seeding the uniform cell's *latency* (n=3) holds mean E2E within ±1.4/2.7/2.0% at λ=12/24/30, so the load-robustness magnitudes are stable too.

Table 10. Multi-seed stability of quality and cost (mean ± s.d. over n=4 Poisson-arrival seeds, headline cells at λ=12).

System (cell, λ=12)	DeepEval quality	cost/req (USD)
RouteBalance w_q=0.8	0.4184±0.0003	6.91e−5
RouteBalance uniform	0.3703±0.0004	2.38e−5
BEST-Route t=0.5, SQ	0.4056±0.0000	3.92e−5
AvengersPro p_w=0.8, SQ	0.3760±0.0000	2.62e−5
Passthrough, random	0.3629±0.0029	4.01e−5

6.7 Judge robustness

The quality of record is one G-Eval judge; to test whether the system ranking is a judge artifact we re-scored the full

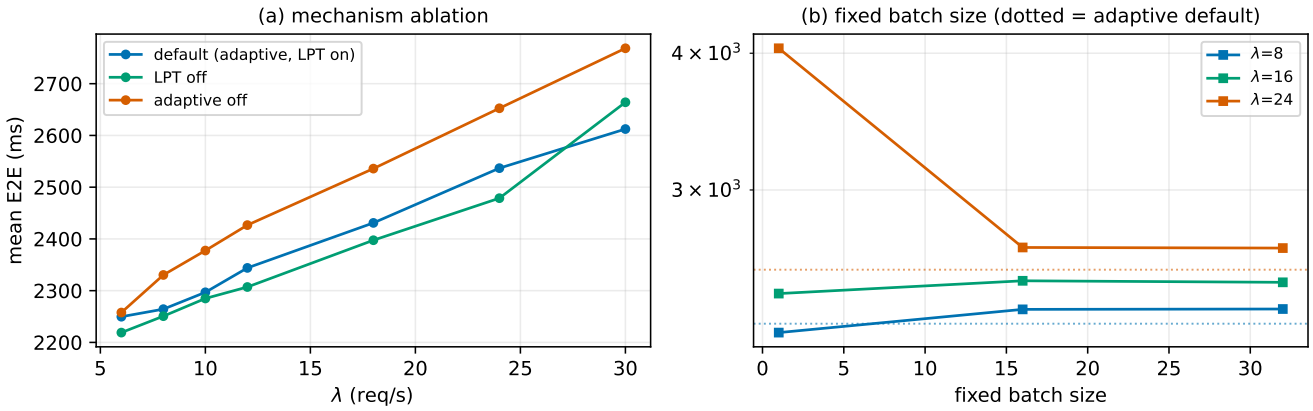

Figure 4. Batching ablation, N=3,534/cell. (a) mean E2E vs. λ for default, LPT-off, and adaptive-off; (b) mean E2E vs. fixed batch size at $\lambda \in \{8, 16, 24\}$.

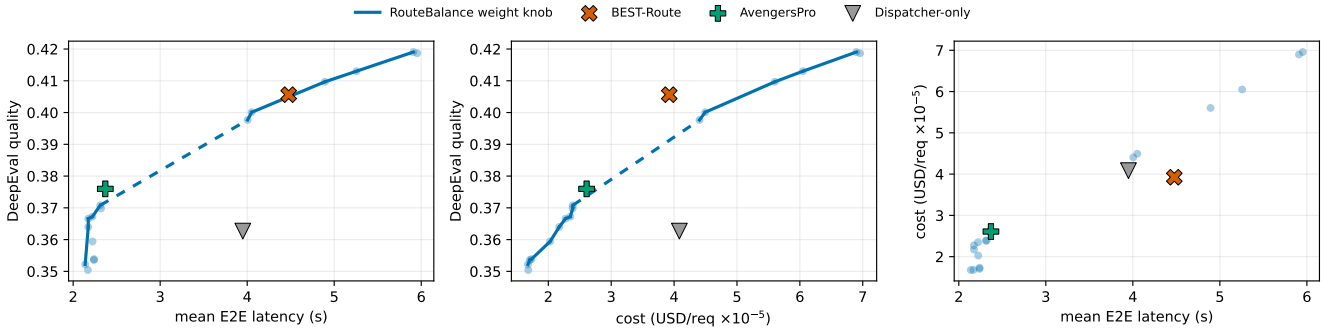

Figure 5. The three pairwise trade-off planes at λ=12 (headline cells; light dots = RouteBalance's other swept cells, line = its frontier in each plane).

(*prompt*, *model*) grid with a second judge, gemma-3-12B-it (n=14,431 pairs, changing only the judge). gemma is uniformly more lenient (per-pair Pearson r=0.555), but RouteBalance leads under both judges: lookup quality RouteBalance 0.696 > BEST-Route 0.684 (+0.011; Llama +0.013) > passthrough 0.642 > Avengers-Pro 0.626. gemma compresses the low corners (passthrough edges Avengers-Pro, reversing Llama), but the RouteBalance>BEST-Route gap is judge-robust. We additionally re-judged the *actual served text* of the headline t=0.5 pair under gemma across two seeds: RouteBalance ranks above BEST-Route in both, with the paired bootstrap placing both deltas above zero (Table 11). Judging served text for *every* cell (>10^6 G-Eval calls) is cost-prohibitive, so we validate it at the headline points; greedy decoding (above) makes the lookup a faithful stand-in elsewhere.

Table 11. Alternate-judge agreement (gemma-3-12B-it vs. Llama-3.1-8B, same G-Eval criteria; §6.7). Top block: second-judge *lookup* over the full grid; lower block: re-judged served text.

	Llama-3.1 judge	gemma-3 judge
RouteBalance w_q=0.8	**0.419**	**0.696**
BEST-Route t=0.5	0.406	0.684
Avengers-Pro p_w=0.8	0.376	0.626
Passthrough (random)	0.363	0.642
Headline-cell re-judge (t=0.5, served text, two seeds)		
RouteBalance w_q=0.8 (s1/s2)	**0.418**	**0.620** / **0.618**
BEST-Route t=0.5/SQ (s1/s2)	0.406	0.600 / 0.605

6.8 Predictor accuracy and headroom

The deployed quality estimator (MiniLM+KNN, k=10) selects DeepEval's best model on 34.8% of held-out prompts (random 25%), and the per-tier XGBoost latency heads achieve

low MAE/MAPE (Table 12). Yet §6.2 shows this already suffices to hold the quality ceiling: the scheduler needs a useful *ranking*, not a calibrated score—the frontier is insensitive to k (over $k \in \{5, 10, 20, 50\}$ routed quality stays within 0.412–0.425). For headroom, an *oracle* (per-prompt argmax of judge scores) reaches 0.582 while a *prompt-blind* mix at RouteBalance's peak-cell tier shares reaches only 0.401, so prompt-dependent selection adds +0.018 at the quality corner.

Table 12. Deployed latency-predictor accuracy on held-out traces (per-(model, GPU) XGBoost; TPOT/TTFT as MAE, end-to-end as MAPE).

Instance type	TPOT MAE (ms/tok)	TTFT MAE (ms)	E2E MAPE	n_{val}
3B / A30	0.22	4.2	2.4%	13,434
7B / A30	0.51	7.6	2.1%	22,465
14B / V100	0.88	8.1	5.6%	13,502
72B / A100	0.75	13.8	1.2%	9,045

Out-of-distribution. A leave-one-dataset-out study (deployed predictor, DeepEval ground truth) tracks in-distribution accuracy within ±0.05 on four of seven folds and *exceeds* it on two (squad +0.09, code +0.05). The one material degradation is gsm8k (0.32→0.23, below the 0.25 random level), a distinct math distribution: no systematic collapse, but a genuinely novel domain can drop the estimator to chance—an honest limit of nearest-neighbor estimation.

Graceful tier loss. Removing the entire 72B tier (both replicas) and re-running the λ=12 cells against the remaining 11 instances degrades gracefully: zero failed requests; the KNN scores re-normalize over the remaining tiers and load redistributes (uniform: 44% 14B / 27% 7B / 29% 3B). Quality falls only to the best-remaining-tier ceiling—the quality cell drops 0.419→0.372, the uniform cell unchanged (0.371→0.371, it used 72B for only 22 of 3,534 requests)—and mean E2E stays bounded (2.9 s). Losing a tier is a capacity/quality-ceiling event, not an availability event.

Safety behavior. Safety-flagged prompts (a 671-prompt subset) follow the same weight-controlled tier policy: under quality-priority they concentrate on 72B (79% vs 51% overall) and reach quality 0.472 (above BEST-Route's 0.396); under cost-priority they shift to 3B like any prompt. No preset routes harmful prompts to systematically worse models—safety follows from the weight vector, not a separate mechanism.

6.9 Tails, non-stationary load, and per-baseline dominance

Tail latency (Table 13) identifies the SLO-safe presets: RouteBalance's *uniform* and *cost* presets keep p95/p99 bounded across the load range, while its *quality-priority* preset is a frontier extreme trading tail latency for the quality ceiling (poor p99 at high load), not meant for tight-latency SLOs. The serial routers' p95/p99 blow up under load; Avengers-Pro holds the p99 rim only at low load. Under bursty and square-wave arrivals (matched mean λ=18) the amortized-scoring systems stay within ~14% of their stationary E2E while the *serial* router degrades up to +74%. A per-axis dominance check across all three pairwise planes (Figure 5; per-system numbers in Table 3) shows RouteBalance wins or ties every axis against every baseline and is itself never strictly dominated: it strictly dominates the decoupled BEST-Route and dispatcher-only baselines on every axis, and against the strongest baseline, Avengers-Pro, wins quality (+0.043) and ties cost and mean latency, the remaining gaps being sub-1% slivers plus Avengers-Pro's low-load p99 tail rim. Cost-model sensitivity (five price vectors) leaves all orderings invariant.

Table 13. Tail latency at the headline operating points (seconds, N=3,534/cell).

	λ=12			λ=24			λ=30		
System	p95	p99	$p99_{TTFT}$	p95	p99	$p99_{TTFT}$	p95	p99	$p99_{TTFT}$
RouteBalance (uniform)	7.9	12.7	0.09	9.0	13.7	0.10	9.2	14.7	0.10
RouteBalance (w_q=0.8)	20.5	43.9	0.13	51.6	76.2	39.9	87.1	111.0	91.3
BEST-Route (t=0.5, SQ)	12.2	20.4	2.3	125.8	134.8	8.8	119.9	127.3	7.4
AvengersPro (p_w=0.8, SQ)	7.5	11.2	0.30	9.2	14.3	1.8	48.6	57.1	7.7
Dispatcher-only (random)	14.5	25.3	0.12	14.7	23.9	0.13	14.5	24.3	0.15

Why latency ties Avengers-Pro. The tie is mechanistic. Both reduce routing to a single sentence-embedding lookup (Avengers-Pro clusters the embedding and reads a precomputed per-cluster ranking [59]; RouteBalance feeds one embedding to its KNN estimator), so neither pays a generative forward per request, unlike BEST-Route's classifier. As published both score one request at a time, saturating the scoring queue under load (Avengers-Pro's k-means residual climbs 258 ms→2.79 s, §6.3); our concurrent-scoring path—a contribution of this work, *not* part of either baseline (§5)—micro-batches the in-flight scoring (routing and quality unchanged), after which mean E2E is serving-bound and both coincide at 2.3–2.8 s. Avengers-Pro reaches RouteBalance's latency only by inheriting its batched-scoring engineering.

7 Conclusion

RouteBalance fuses model routing and load balancing into a single online assignment over concrete model instances on a quality–latency–cost simplex; one deployed stack spans cheapest-to-highest-quality by changing only the weight vector, and a four-arm decomposition traces the benefit to *pricing latency at model-selection time*, a decision the decoupled router-then-balancer stack cannot make. The gains hold at scale (28 GPUs, 442 configurations, ≈1.5M requests) against engineering-equalized baselines, with judge-, seed-, and cost-model-robust orderings. Open directions: a second model family and topology (a Llama/Gemma port is the immediate next step), production-trace arrivals, and an SLO-driven controller over the weights. The framework, scripts, and datasets are released for artifact evaluation.

References

[1] Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. Vidur: A large-scale simulation framework for llm inference. *ArXiv abs/2405.05465* (2024).

[2] Ahmad, S., Guan, H., Friedman, B. D., Williams, T., Sitaraman, R. K., and Woo, T. Proteus: A high-throughput inference-serving system with accuracy scaling. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 1* (2024), pp. 318–334.

[3] Arango, I., Noori, A., Huang, Y., Shahout, R., and Yu, M. Prefix and output length-aware scheduling for efficient online LLM inference. In *Sparsity in LLMs (SLLM): Deep Dive into Mixture of Experts, Quantization, Hardware, and Inference* (2025).

[4] Bao, R., Xue, N., Sun, Y., and Chen, Z. Dynamic quality-latency aware routing for llm inference in wireless edge-device networks, 2025.

[5] Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., and Molchanov, P. Small language models are the future of agentic ai. *arXiv preprint arXiv:2506.02153* (2025).

[6] Bin, K., Choi, S., Son, J., Choi, J., Bae, D., Baek, D., Moon, K., Jang, M., and Lee, H. Fineserve: Precision-aware kv slab and two-level scheduling for heterogeneous precision llm serving, 2025.

[7] Chang, T.-T., and Venkataraman, S. Eva: Cost-efficient cloud-based cluster scheduling. In *Proceedings of the Twentieth European Conference on Computer Systems* (New York, NY, USA, 2025), EuroSys '25, Association for Computing Machinery, p. 1399–1416.

[8] Chen, L., Zaharia, M., and Zou, J. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176* (2023).

[9] Chen, S., Jia, Z., Khan, S., Krishnamurthy, A., and Gibbons, P. B. Slos-serve: Optimized serving of multi-slo llms, 2025.

[10] Chen, T., and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (2016), pp. 785–794.

[11] Cho, J., Kim, M., Choi, H., Heo, G., and Park, J. LLMServingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale . In *2024 IEEE International Symposium on Workload Characterization (IISWC)* (Los Alamitos, CA, USA, Sept. 2024), IEEE Computer Society, pp. 15–29.

[12] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021.

[13] Da, W., and Kalyvianaki, E. Block: Balancing load in llm serving with context, knowledge and predictive scheduling, 2025.

[14] Dexter, G., Tang, S., Fatahibaarzi, A., Song, Q., Dharamsi, T., and Gupta, A. LLM query scheduling with prefix reuse and latency constraints. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems* (2026).

[15] Ding, D., Mallick, A., Zhang, S., Wang, C., Madrigal, D., Garcia, M. D. C. H., Xia, M., Lakshmanan, L. V. S., Wu, Q., and Rühle, V. BEST-route: Adaptive LLM routing with test-time optimal compute. In *Forty-second International Conference on Machine Learning* (2025).

[16] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library. *IEEE Transactions on Big Data* (2025).

[17] Duplyakin, D., Ricci, R., Maricq, A., Wong, G., Duerig, J., Eide, E., Stoller, L., Hibler, M., Johnson, D., Webb, K., Akella, A., Wang, K., Ricart, G., Landweber, L., Elliott, C., Zink, M., Cecchet, E., Kar, S., and Mishra, P. The design and operation of CloudLab. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)* (Renton, WA, July 2019), USENIX Association, pp. 1–14.

[18] Fan, Q., Zou, A., and Ma, Y. Timebill: Time-budgeted inference for large language models, 2025.

[19] Fang, J., Shen, Y., Wang, Y., and Chen, L. Improving the end-to-end efficiency of offline inference for multi-llm applications based on sampling and simulation, 2025.

[20] Graham, R. L. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics 17*, 2 (1969), 416–429.

[21] Gunasekaran, J. R., Mishra, C. S., Thinakaran, P., Sharma, B., Kandemir, M. T., and Das, C. R. Cocktail: A multidimensional optimization for model serving in cloud. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (2022).

[22] Hu, Q. J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., and Upadhyay, S. K. Routerbench: A benchmark for multi-llm routing system. *arXiv preprint arXiv: 2403.12031* (2024).

[23] Ji, J., Liu, M., Dai, J., Pan, X., Zhang, C., Bian, C., Chen, B., Sun, R., Wang, Y., and Yang, Y. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. In *Advances in Neural Information Processing Systems* (2023).

[24] Jiang, D., Ren, X., and Lin, B. Y. Llm-blender: Ensembling large language models with pairwise ranking and generative fusion. In *Annual Meeting of the Association for Computational Linguistics (ACL)* (2023).

[25] JIANG, Y., Fu, F., Yao, X., HE, G., Miao, X., Klimovic, A., CUI, B., Yuan, B., and Yoneki, E. Demystifying cost-efficiency in LLM serving over heterogeneous GPUs. In *Forty-second International Conference on Machine Learning* (2025).

[26] JIANG, Y., Fu, F., Yao, X., Wang, T., CUI, B., Klimovic, A., and Yoneki, E. Thunderserve: High-performance and cost-efficient LLM serving in cloud environments. In *Eighth Conference on Machine Learning and Systems* (2025).

[27] Jiang, Y., Yan, R., Yao, X., Zhou, Y., Chen, B., and Yuan, B. Hexgen: generative inference of large language model over heterogeneous environment. In *Proceedings of the 41st International Conference on Machine Learning* (2024), ICML'24, JMLR.org.

[28] JIANG, Y., Yan, R., and Yuan, B. Hexgen-2: Disaggregated generative inference of LLMs in heterogeneous environment. In *The Thirteenth International Conference on Learning Representations* (2025).

[29] Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, C., Wang, Z., Go, A., Lee, C.-Y., Shenoy, P., Panigrahy, R., et al. Universal model routing for efficient llm inference. *arXiv preprint arXiv:2502.08773* (2025).

[30] Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, C., Wang, Z., Go, A., Lee, C.-Y., Shenoy, P., Panigrahy, R., Menon, A. K., and Kumar, S. Universal model routing for efficient LLM inference. In *The Fourteenth International Conference on Learning Representations* (2026).

[31] Kraska, T., Beutel, A., Chi, E. H., Dean, J., and Polyzotis, N. The case for learned index structures. In *Proceedings of the 2018 International Conference on Management of Data* (New York, NY, USA, 2018), SIGMOD '18, Association for Computing Machinery, p. 489–504.

[32] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (New York, NY, USA, 2023), SOSP '23, Association for Computing Machinery, p. 611–626.

[33] Lambert, N., Pyatkin, V., Morrison, J., Miranda, L., Lin, B. Y., Chandu, K., Dziri, N., Kumar, S., Zick, T., Choi, Y., Smith, N. A., and Hajishirzi, H. Rewardbench: Evaluating reward models for language modeling, 2024.

[34] Li, Y. Rethinking predictive LLM routing: When simple KNN beats complex learned routers, 2026.

[35] Mai, L., et al. Exploiting replication for energy-efficient large-scale parallel batch scheduling. City Research Online, 2013.

[36] Mei, K., Xu, W., Guo, M., Lin, S., and Zhang, Y. Omnirouter: Budget and performance controllable multi-llm routing. *SIGKDD Explor. Newsl. 27*, 2 (Dec. 2026), 107–116.

[37] Mei, Y., Zhuang, Y., Miao, X., Yang, J., Jia, Z., and Vinayak, R. Helix:

Serving large language models over heterogeneous gpus and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (New York, NY, USA, 2025), ASPLOS '25, Association for Computing Machinery, p. 586–602.

[38] Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665* (2024).

[39] Panda, P., Magazine, R., Devaguptapu, C., Takemori, S., and Sharma, V. Adaptive llm routing under budget constraints. *arXiv preprint arXiv:2508.21141* (2025).

[40] Patke, A., Reddy, D., Jha, S., Qiu, H., Pinto, C., Narayanaswami, C., Kalbarczyk, Z., and Iyer, R. Queue management for slo-oriented large language model serving, 2025.

[41] Qin, R., Li, Z., He, W., Cui, J., Tang, H., Ren, F., Ma, T., Cai, S., Zhang, Y., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Trans. Storage* (Nov. 2025). Just Accepted.

[42] Qwen Team, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025.

[43] Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. SQuAD: 100,000+ questions for machine comprehension of text. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2016).

[44] Reddy, T. R., Deshmukh, A., Tandon, K., Gandhi, R., Parayil, A., and Bhattacherjee, D. Bellman: Controlling llm congestion, 2025.

[45] Romero, F., Li, Q., Yadwadkar, N. J., and Kozyrakis, C. INFaaS: Automated model-less inference serving. In *USENIX Annual Technical Conference (ATC)* (2021).

[46] Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. Llumnix: dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (USA, 2024), OSDI'24, USENIX Association.

[47] Sun, T., Wang, P., and Lai, F. Hygen: Efficient LLM serving via elastic online-offline request co-location. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems* (2026).

[48] Tao, Y., Zhang, Y., Dearing, M. T., Wang, X., Fan, Y., and Lan, Z. Prompt-aware scheduling for low-latency llm serving, 2025.

[49] Team, D. DeepEval: an LLM evaluation framework. https://github.com/confident-ai/deepeval, 2024.

[50] Tian, J., Li, S., Cao, Y., Cui, W., Zhu, M., Wu, W., Zhang, J., Wang, Y., Xiao, Z., Hou, Z., and Shen, D. Staggered batch scheduling: Co-optimizing time-to-first-token and throughput for high-efficiency llm inference, 2025.

[51] Wang, C., Liu, X., Liu, Y., Zhu, Y., Mo, X., Jiang, J., and Chen, H. When to reason: Semantic router for vllm. *arXiv preprint arXiv:2510.08731* (2025).

[52] Wang, Y., Chen, K., Tan, H., and Guo, K. Tabi: An efficient multi-level inference system for large language models. In *European Conference on Computer Systems (EuroSys)* (2023).

[53] Wen, H., Wu, X., Sun, Y., Zhang, F., Chen, L., Wang, J., Liu, Y., Liu, Y., Zhang, Y.-Q., and Li, Y. Budgetthinker: Empowering budget-aware llm reasoning with control tokens, 2025.

[54] Weyssow, M., Kamanda, A., and Sahraoui, H. Codeultrafeedback: An llm-as-a-judge dataset for aligning large language models to coding preferences, 2024.

[55] Wu, F., and Silwal, S. PORT: Efficient training-free online routing for high-volume multi-LLM serving. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems* (2025). arXiv:2509.02718.

[56] Wu, Z., Markakis, M., Liu, C., Chen, P. B., Narayanaswamy, B., Kraska, T., and Madden, S. Improving dbms scheduling decisions with accurate performance prediction on concurrent queries. *Proc. VLDB Endow. 18*, 11 (July 2025), 4185–4198.

[57] Xu, T., Liu, Y., Lu, X., Zhao, Y., Zhou, X., Feng, A., Chen, Y., Shen, Y., Zhou, Q., Chen, X., Sherstyuk, I., Li, H., Thakkar, R., Hamm, B., Li, Y., Huang, X., Wu, W., Shanbhag, A., Kim, H., Chen, C., and Lai, J. Aiconfigurator: Lightning-fast configuration optimization for multi-framework llm serving, 2026.

[58] Yuan, Y., Zhao, C., Zhao, B., Cao, Z., He, Y., and Wu, W. Cascade-infer: Low-latency and load-balanced llm serving via length-aware scheduling. *arXiv preprint arXiv:2512.19179* (2025).

[59] Zhang, Y., Li, H., Chen, J., Zhang, H., Ye, P., Bai, L., and Hu, S. Beyond gpt-5: Making llms cheaper and better via performance-efficiency optimized routing. In *Proceedings of the 2025 7th International Conference on Distributed Artificial Intelligence* (New York, NY, USA, 2025), DAI '25, Association for Computing Machinery, p. 122–129.

[60] Zhao, Z., Hu, Y., Chen, S., Ji, M., Yang, W., Zhang, Y., Zhao, L., Li, W., Liu, X., Qu, W., and Wang, H. Pard: Enhancing goodput for inference pipeline via proactive request dropping. In *Proceedings of the 21st European Conference on Computer Systems* (New York, NY, USA, 2026), EUROSYS '26, Association for Computing Machinery, p. 423–438.

[61] Zheng, L., Chiang, W.-L., Sheng, Y., Li, T., Zhuang, S., Wu, Z., Zhuang, Y., Li, Z., Lin, Z., Xing, E. P., Gonzalez, J. E., Stoica, I., and Zhang, H. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on Learning Representations (ICLR)* (2024).

[62] Zheng, W., Xu, M., Song, S., and Ye, K. Bucketserve: Bucket-based dynamic batching for smart and efficient llm inference serving, 2025.

[63] Zheng, Z., Ren, X., Xue, F., Luo, Y., Jiang, X., and You, Y. Response length perception and sequence scheduling: An LLM-empowered LLM inference pipeline. In *Thirty-seventh Conference on Neural Information Processing Systems* (2023).

[64] Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024.

[65] Zhu, K., Shi, H., Xu, L., Shan, J., Krishnamurthy, A., Kasikci, B., and Xie, L. Polyserve: Efficient multi-slo serving at scale, 2025.